

Contents

Defense Study Guide — PMSM Fault Diagnosis (Scalograms + CNN)	1
1. The 60-second elevator pitch	1
2. Core concepts (be able to explain each in your own words)	2
2.1 PMSM motor	2
2.2 The faults	2
2.3 Why vibration can beat current	2
2.4 Fourier vs Wavelet	2
2.5 CWT and the Morlet wavelet	2
2.6 Scalogram	2
2.7 CNN	3
2.8 Metrics & pitfalls	3
2.9 Data leakage & the split	3
2.10 Class imbalance handling	3
3. Project specifics & numbers (memorize these)	3
4. Defense Q&A bank (anticipated questions + strong answers)	4
5. “Must be able to explain at the whiteboard” checklist	5
6. Likely “trap” questions (and the honest answer)	6
7. Where to find things (for live demo during defense)	6

Defense Study Guide — PMSM Fault Diagnosis (Scalograms + CNN)

Everything you need to understand the project deeply and defend it confidently. Study in this order: (1) the 60-second pitch, (2) core concepts, (3) the project specifics & numbers, (4) the Q&A bank, (5) the “must be able to explain” checklist.

1. The 60-second elevator pitch

“PMSM motors develop stator faults — most commonly an inter-turn short — that can destroy the winding. We detect them automatically. We take the motor’s current and vibration signals, cut them into half-second windows, and turn each window into a **wavelet scalogram** — a time-frequency image that shows how the signal’s energy is distributed across frequencies over time. A **CNN** then classifies each image as healthy or faulty. On the real KAIST dataset, the **vibration channel reaches perfect balanced accuracy (1.00)** on unseen recordings, while the **current channel is much weaker (0.69)** — showing vibration is the better sensor for this fault. The whole pipeline is in Python, reproducible, tested, and we report balanced accuracy rather than raw accuracy because the data is imbalanced.”

2. Core concepts (be able to explain each in your own words)

2.1 PMSM motor

A **Permanent Magnet Synchronous Motor**: rotor has permanent magnets, stator has a 3-phase winding. The 3-phase current makes a rotating magnetic field; the rotor locks to it and spins **synchronously** (no slip, no rotor current → high efficiency). Controlled by **Field-Oriented Control (FOC)**, which splits current into i_d (flux, kept ~ 0) and i_q (torque). Energy balance: $v \cdot i \approx T \cdot \omega$ (voltage ↔ speed, current ↔ torque).

2.2 The faults

- **Inter-turn short (ITSC)** — insulation fails between turns of the same phase; a big circulating current heats up and worsens. *Most common, our main fault.*
- **Demagnetization** — rotor magnets weaken (heat/age) → sub-harmonics.
- **Overload** — running above rated load → raised fundamental + harmonics.
- (Real data has Healthy + Inter-turn; the other two are synthetic only.)

2.3 Why vibration can beat current

The FOC controller actively **suppresses** disturbances in the current (it's a closed loop regulating current), partly masking the fault signature. Vibration is an open, direct mechanical response to the fault → cleaner signature. Our results confirm this.

2.4 Fourier vs Wavelet

- **Fourier transform**: tells you *which* frequencies exist, but not *when* — it's "blind to time". Fine for **stationary** signals.
- Fault signals are **non-stationary** (transient, load-dependent).
- **Uncertainty principle**: $\Delta t \cdot \Delta f \geq \text{constant}$ — you can't have perfect time *and* frequency resolution simultaneously.
- **Wavelet transform** uses short, localized "little waves" and gives **frequency-dependent resolution**: good frequency resolution at low frequencies, good time resolution at high frequencies — ideal for transients.

2.5 CWT and the Morlet wavelet

The **Continuous Wavelet Transform** slides (time) and scales (frequency) a mother wavelet across the signal; each coefficient $T(a,b)$ is the **similarity** (a dot product) between the signal and that scaled/shifted wavelet. We use the **complex Morlet** wavelet = a cosine times a Gaussian bell (cmor1.5-1.0), over 128 scales. A wavelet must have **zero mean** (admissibility) and **finite energy**.

2.6 Scalogram

The **image of |CWT|**: x = time, y = scale/frequency, color = energy. Bright bands/sidebands are the visible fault signatures. We render 224×224 RGB PNGs. This converts a *signal* problem into an *image* problem.

2.7 CNN

A Convolutional Neural Network learns spatial patterns in images automatically: - **Conv layer**: small learnable filters slide over the image → feature maps (edges, bands, textures). - **ReLU**: sets negatives to 0 → non-linearity. - **Max-pool**: downsamples, keeps strongest responses, adds translation invariance. - **Flatten / Global Average Pooling**: 2-D maps → 1-D vector. - **Dense + Softmax**: final decision → class probabilities. Key advantages: **parameter sharing** (same filter everywhere) and preserving **spatial structure**. Our net: 3 conv blocks (32/64/128) → dropout → dense → softmax.

2.8 Metrics & pitfalls

- **Accuracy** = fraction correct — *misleading on imbalanced data* (predict the majority class and you look good).
- **Recall** (per class) = of the true X, how many did we catch.
- **Precision** = of those we called X, how many were right.
- **F1** = harmonic mean of precision & recall; **macro-F1** = average over classes.
- **Balanced accuracy** = mean of per-class recalls — our headline metric.
- **Confusion matrix** = table of true vs predicted.

2.9 Data leakage & the split

If windows from the same recording appear in both train and test, the model “memorizes” the recording → inflated scores. We prevent this by splitting **by recording** (whole recordings go to one split), stratified per class, per channel.

2.10 Class imbalance handling

Only ~4 healthy vs ~60 fault recordings. We **undersample the majority in train/val**, keep **test natural**, and report **balanced accuracy / macro-F1**.

3. Project specifics & numbers (memorize these)

- **Dataset**: KAIST, Mendeley rgn5brrgrn, CC-BY-4.0. Current @ 100 kHz, vibration @ 25.6 kHz, TDMS files; normal / inter-turn / inter-coil.
- **Preprocessing**: decimate to **10 kHz**; **0.5 s** windows, **50 %** overlap; cap **50 segments/recording**; **3,150** scalograms total (1,550 current + 1,600 vibration).
- **Wavelet**: complex Morlet cmor1.5-1.0, **128 scales** (4→256 geospace), **224×224** RGB.
- **Split**: 70/15/15 by recording, stratified, per channel, leakage-free.
- **Model**: 3 conv blocks (32/64/128) + dropout 0.5 + dense 128; Adam; sparse categorical cross-entropy; early stopping (patience 5); random-flip augmentation.
- **Results (held-out)**: vibration **bal-acc 1.00**; current **0.69**; fusion **0.88**.
- **Ablations**: balancing → healthy recall 0→1.00; image size 96→224 helps current (0.68→0.76), vibration saturated; learning curve: vibration needs ~46 images, current flat ~0.70.
- **Limitation**: only **4 distinct healthy recordings**.
- **Engineering**: 38 unit tests, CI on Py 3.10–3.12, GPU (Quadro P620, ~17×).

- **Tools:** Python — PyWavelets (CWT), TensorFlow/Keras (CNN), npTDMS, scipy, scikit-learn. MATLAB is optional only.
-

4. Defense Q&A bank (anticipated questions + strong answers)

Q1. Why scalograms instead of feeding the raw signal to the network? Raw 1-D signals hide fault signatures in subtle time-frequency structure. Scalograms expose that structure as visual patterns and let us use CNNs, which are extremely good at images. It also makes the model robust to small shifts.

Q2. Why wavelets and not FFT / spectrogram? FFT is blind to time and assumes stationarity; fault signals are non-stationary. A spectrogram (STFT) has a single fixed time/frequency trade-off. The wavelet transform adapts resolution per frequency, which suits transients — good time localization at high frequency, good frequency localization at low frequency.

Q3. Why the Morlet wavelet specifically? It's a Gaussian-windowed complex sinusoid — the standard choice for oscillatory signals; its magnitude gives a smooth energy estimate, and it satisfies the admissibility (zero-mean) and finite-energy conditions.

Q4. Your vibration accuracy is 1.00 — isn't that suspicious / overfitting? We were careful: the split is **leakage-free by recording**, and 1.00 is on **held-out recordings**, not random windows. The honest caveat is that there are only **4 healthy recordings**, so we can't fully exclude the model keying on recording-specific characteristics. That's why we state it as a limitation and recommend collecting more independent healthy recordings. Physically, vibration is expected to be a strong channel for inter-turn faults.

Q5. Why is current so much worse than vibration? Two reasons: (a) the inter-turn current signature is weak at the available severities; (b) the FOC current controller actively suppresses disturbances in the current. Vibration responds directly to the fault. Our learning-curve experiment shows more current data doesn't help (flat ~0.70) → it's a signal-quality limit, not a data-quantity limit.

Q6. Why report balanced accuracy instead of accuracy? The test set is 50 healthy / 200 faulty. A model that always says "faulty" gets 80 % accuracy but never detects a healthy motor — useless. Balanced accuracy (mean of per-class recalls) and macro-F1 expose that immediately.

Q7. What is data leakage and how did you avoid it? If near-identical windows from one recording land in both train and test, the score is inflated. We split by **recording_id** so an entire recording is in only one split, stratified by class, separately per channel.

Q8. How did you handle the class imbalance? Undersample the majority class in **train/val only**; keep the **test set natural**. We proved (ablation) that without this the current model collapses to predicting "fault" (0 % healthy recall).

Q9. What does the fusion model do and why didn't it win? It has two conv branches (current + vibration) merged before the classifier. It beat current alone (0.88 vs 0.69) but not vibration alone (1.00) — adding a weak channel to a strong one didn't help on this small dataset. With a harder problem it should help.

Q10. Why 0.5 s windows and 10 kHz? 0.5 s captures several electrical cycles (50 Hz → 25 cycles) — enough for frequency content. We decimate 100 kHz → 10 kHz to keep scalograms a manageable size while keeping all fault-relevant frequencies (Nyquist 5 kHz).

Q11. How many parameters / how big is the model? Did it overfit? A small 3-block CNN, deliberately sized for a few-thousand-image dataset. Overfitting controls: dropout 0.5, data augmentation, early stopping, global average pooling in the fusion head. The flat current learning curve confirms capacity isn't the bottleneck.

Q12. Do you still need MATLAB? No. Everything runs in Python (PyWavelets + TensorFlow). MATLAB scripts are an optional alternative path; no reported result depends on them.

Q13. How is it reproducible? Fixed seed (42) everywhere; one config.yaml; one manifest.csv; 38 unit tests; CI on every push; documented commands in the README.

Q14. What would you do with more time / future work? Collect more independent healthy recordings; build a synchronized multi-channel dataset so fusion can shine; predict fault **severity** (regression); add **Grad-CAM** explainability to confirm the network looks at physically meaningful regions; deploy on-line inside the drive.

Q15. What is inter-turn vs inter-coil? Both are stator winding short circuits; inter-turn is between turns of the same coil, inter-coil between coils. We grouped both as the "InterTurn" fault class.

Q16. Why CNN and not a classic ML classifier (SVM, random forest)? Those need hand-crafted features; the CNN learns features directly from the scalogram image. Given the signatures are spatial/visual, CNNs are the natural fit.

Q17. What's on the scalogram axes, and why is it blurry at the bottom? X = time, Y = scale ($\approx$ inverse frequency), color = energy. The "blur" reflects the uncertainty principle: low frequencies (bottom) have fine frequency but coarse time resolution; high frequencies (top) are the opposite.

Q18. How did you validate the software itself? Synthetic data (separable by construction) gives 100 % — that proves the *pipeline* is wired correctly end-to-end (no leakage, correct labels/metrics), separately from the *difficulty* of the real problem.

5. "Must be able to explain at the whiteboard" checklist

- ☐ Draw the full pipeline: signal → window → CWT → scalogram → CNN → class.
- ☐ Sketch a wavelet (Morlet) and explain scale vs shift.
- ☐ Explain the uncertainty principle and the Heisenberg boxes on a scalogram.
- ☐ Draw the CNN layer stack and say what each layer does.
- ☐ Define accuracy, recall, precision, F1, balanced accuracy; compute a small confusion matrix by hand.
- ☐ Explain data leakage and the recording-level split with a drawing.
- ☐ Explain the imbalance fix and why the test set stays natural.
- ☐ State the headline numbers (vibration 1.00, current 0.69, fusion 0.88) and the 4-healthy-recordings limitation.

- Justify every config value (0.5 s, 10 kHz, 128 scales, 224 px, 50/15/15...).
-

6. Likely “trap” questions (and the honest answer)

- “*So your model is 100 % accurate?*” → “Balanced accuracy 1.00 on the vibration channel on held-out recordings — but only 4 healthy recordings exist, so we present it with that limitation, not as a finished product.”
 - “*Isn’t synthetic 100 % cheating?*” → “It only validates the software path; we never present it as a real result.”
 - “*Why not just use current — it’s cheaper?*” → “We tested exactly that; current is too weak for this fault at these severities. That’s a finding, not a gap.”
-

7. Where to find things (for live demo during defense)

- Run tests: `make test` (38 pass).
- Synthetic end-to-end: `make demo`.
- Figures: `results/` (`confusion_real_2class.png`, `example_scalograms_real.png`, `learning_curve.png`).
- Report/slides: `docs/build/` (PDF/DOCX/PPTX, EN + AR).
- Code map: `docs/build-walkthrough.md`.
- Repo: github.com/molhamfetnah/pmsm-fault-diagnosis-cnn-scalogram.